# Report – Run multi-seed HalfMamba / HalfMamba_Augmented

19/10/2026

Fonte: `train/run_all_seeds.py`, `train/metrics/`

## Sintesi

- **HalfMamba_Augmented batte HalfMamba su tutti e 3 i seed**: val loss $-4\%$ circa, con errori su $p$ e $v_y$ più bassi. Il costo è $+33\%$ di tempo per epoca e $+50\%$ di VRAM, con solo $\sim$40k parametri in più.
- **La variante "merged" (Comb. 4 + Comb. 12) è la migliore in media** e ha la varianza più bassa sulla loss. Il vantaggio su Aug. exp1 è però **dentro il rumore** tra seed.
- **I valori del tuning a seed singolo non si ripetono**: Comb. 4 con seed 42 passa da 0,00300 (tuning) a 0,00318 (questo run), con stessa config e stesso seed. La causa è il non-determinismo del training su GPU.
- **Nessun test set**: il run è stato lanciato con `RUN_TEST = 0`, quindi ci sono solo metriche di validazione.

## 1 Setup

- Dataset small, GPU, 150 epoche per run, seed {42, 1955, 2003}.
- Iperparametri Mamba come `d_state-ssm_ratio-mlp_ratio` per blocco (`out1`, `out2`, `out2b`, decoder):
  - **HalfMamba exp1**: `16-2-4 / 16-2-4 / 16-2-4` (default).
  - **Aug. exp1** (Combination 4 del tuning): `16-2-4 / 16-2-4 / 16-2-4 / 32-2-4`.
  - **Aug. merged** (Comb. 4 + Comb. 12): `16-2-4 / 24-2-2 / 16-2-4 / 32-2-4`.
- Metriche prese all'**ultima epoca** di `metrics_train.csv`. La colonna "Val err (20)" è la media dell'errore relativo sulle ultime 20 epoche, la stessa metrica usata in `runTuning.py`.

## 2 Risultati aggregati

Media $\pm$ deviazione standard sui 3 seed. Per loss, accuratezza ed errori per campo la std è di popolazione (ddof $= 0$, come nei file `summary_*.txt`); per Val err (20) è campionaria (ddof $= 1$).

| | Baseline | HalfMamba | Aug. exp1 | Aug. merged |
|---|---|---|---|---|
| Parametri | 2,33 M | 2,54 M | 2,58 M | 2,57 M |
| Val loss | 0,003654 | 0,003525 | 0,003387 | **0,003332** |
| | ±0,000017 | ±0,000143 | ±0,000154 | ±0,000079 |
| Val accuracy | 98,07% | 98,14% | 98,21% | **98,24%** |
| Val err (20) | 1,953%* | 1,884 ± 0,087% | 1,800 ± 0,084% | **1,784 ± 0,049%** |
| Err. $p$ | 9,60% | 9,34 ± 0,17% | 9,01 ± 0,14% | **8,93 ± 0,45%** |
| Err. $v_x$ | 1,10% | 1,08 ± 0,05% | 1,03 ± 0,05% | **1,01 ± 0,02%** |
| Err. $v_y$ | 4,31% | 4,01 ± 0,04% | 3,89 ± 0,11% | **3,81 ± 0,12%** |
| Tempo / seed | – | ∼57 min | ∼76 min | ∼77 min |
| VRAM picco | – | ∼490 MB | ∼745 MB | ∼740 MB |

* valore di riferimento usato in `runTuning.py`.

## 3 Risultati per seed

| Val loss finale | HalfMamba | Aug. exp1 | Aug. merged |
|---|---|---|---|
| Seed 42 | 0,00333 | **0,00318** | 0,00325 |
| Seed 1955 | 0,00365 | 0,00343 | **0,00331** |
| Seed 2003 | 0,00360 | 0,00355 | **0,00344** |

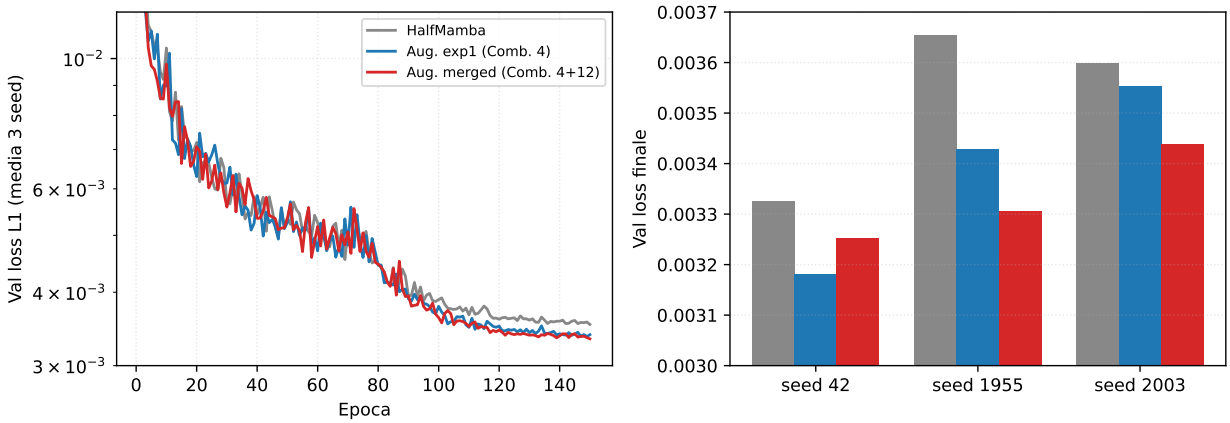



Figura 1: Sinistra: val loss media sui 3 seed per epoca (scala log). Destra: val loss finale per seed.

## 4 Osservazioni

1. **Augmented vs HalfMamba**: il miglioramento è consistente su tutti i seed, quindi è un effetto reale.
2. **Merged vs exp1**: merged vince su 1955 e 2003 e perde su 42. La differenza media (0,000055 di loss, 0,016 punti di val err) è molto più piccola della std tra seed. Le due configurazioni sono di fatto equivalenti; merged è leggermente più stabile sulla loss.
3. **Errore su $p$ instabile in merged**: 8,54% e 8,68% sui seed 1955 e 2003, ma 9,56% sul 42.
4. **Il seed 42 dà la val loss più bassa** per tutti e tre i modelli, anche perché con `split_seed = None` il validation set cambia con il seed.
5. **Modelli non del tutto a convergenza**: la val loss minima cade tra l'epoca 131 e la 150, spesso all'ultima.

## 5 Confronto con il tuning a seed singolo

| Comb. 4, seed 42 | Val loss finale | Val err (20) | Err. $p$ |
|---|---|---|---|
| Tuning (`Tuning_all_layers`) | 0,00300 | 1,628% | 8,66% |
| Questo run (`exp1_s42`) | 0,00318 | 1,711% | 9,20% |

Stesso script, configurazione, seed e split: le curve coincidono quasi all'epoca 1 e poi divergono gradualmente. È non-determinismo su GPU (`cudnn.benchmark = True`, kernel di scan selettivo di Mamba con operazioni atomiche). Questa variabilità (circa 0,08 punti di val err) è paragonabile all'intervallo coperto dalle 15 combinazioni del tuning (1,63–1,86%). Scegliere la migliore tra 15 run a seed singolo seleziona anche la più fortunata, e alla ripetizione il valore torna verso la media.

## 6 Prossimi passi

- Lanciare il test (`runTest.py`) sui 9 modelli per confrontarli con la baseline sul test set.
- Fissare `split_seed = 42` in `runTrain.py`, così tutti i seed usano lo stesso validation set.
- Nel tuning, ripetere su più seed almeno le prime 3–4 combinazioni prima di sceglierne una.